CSCI 2406 | Computer Organization & Architecture | TOPIC 9

Machine Code & AArch64 Instruction Encoding

Instruction words, encoding fields, immediates, memory operations, and branch targets in AArch64

Dr. J. Burns

Topic 9 Outline

1. Machine Code and 32-bit Instruction Words
2. AArch64 Encoding Families
3. Data-Processing Encodings
4. Memory Instruction Encodings
5. Branch Instruction Encodings

Section 1

Machine Code and 32-bit Instruction Words

From Assembly to Machine Code

- A processor repeatedly **fetches**, **decodes**, and **executes** instructions.
- During fetch–decode–execute, the processor fetches a 32-bit instruction word from memory and interprets its bit fields.
- This topic starts with the earlier step: how an AArch64 assembly instruction is encoded as that fixed **32-bit** machine instruction.



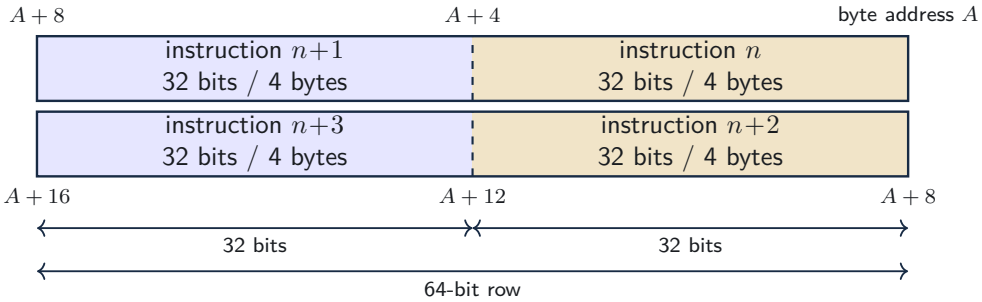
In memory, `0x8B020020` is stored as bytes `20 00 02 8B` in increasing address order (A64 instruction byte order).

AArch64 Encoding Essentials

- Every A64 instruction is a 32-bit word containing operation and operand fields.
- Different instruction families use different field layouts.
- Register fields are normally 5 bits, encoding register numbers 0–31.
- There are 31 general-purpose registers. Register number 31 denotes the zero register or stack pointer, depending on the instruction and operand.
- Identify the instruction form before interpreting its fields.

AArch64 Instruction Word Basics

Each AArch64 instruction is 32 bits, or 4 bytes. Four consecutive instructions occupy 16 bytes, shown here as two 8-byte rows.



Instruction Boundaries

Each instruction starts 4 bytes after the previous one. A 64-bit memory row holds two AArch64 instructions in this diagram; it is not a required hardware fetch width.

Section 2

AArch64 Encoding Families

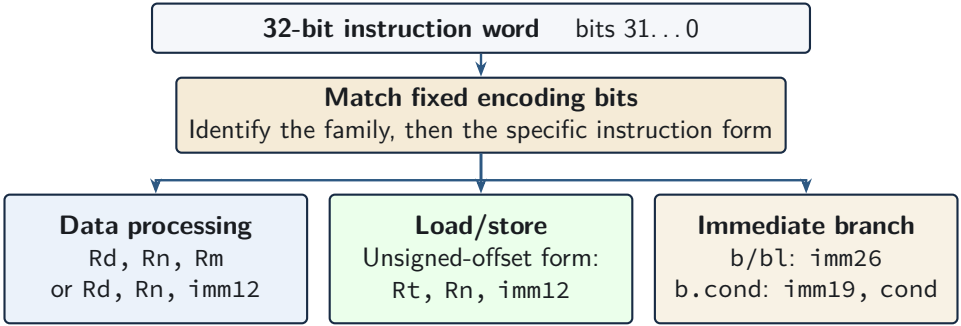
Where Do Opcode Patterns Come From?

- **Arm assigns the encoding templates.** Look up the fixed family/form bits in the ISA; they cannot be calculated from the mnemonic.
- **Select the form first:** `add x5, x6, x7` uses a shifted-register form; `add x5, x6, #24` uses an immediate form with a different template.
- **Fill the selectors:** width, operation, flag update, shift type, access size, or addressing mode choose among related encodings.
- **Insert the operands:** register numbers, immediates, and offsets occupy the remaining fields.

A Prefix Is Not Necessarily One Opcode Field

Leading bits may combine fixed family bits with selectors. Each table states the form and bit positions to which its patterns apply.

From Instruction Word to Operand Fields



The Form Determines the Meaning of the Fields

There is no universal opcode layout. The selected form determines how to interpret register numbers, immediates, shifts and offsets. The following slides expand these examples.

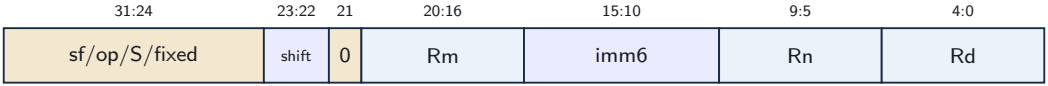
Section 3

Data-Processing Encodings

ADD/SUB: Shifted-Register Form

Example Instruction

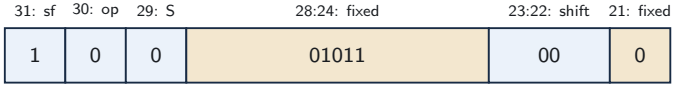
```
add x5, x6, x7
```



- `sf/op/S/fixed` will be defined on the next slide.
- `Rd` is the destination register.
- `Rn` is the first source register.
- `Rm` is the second source register.
- Bits 23:22 select the shift type; `imm6` (15:10) gives the shift amount. Bit 21 is fixed at 0.

Deriving the Prefix: `add x5, x6, x7`

Select **ADD/SUB (shifted register)**. Bits 31:21 have this structure:



Field	Choice	Reason
`sf`	`1`	64-bit `x` operands; `0` selects 32-bit `w` operands
op	`0`	Add; `1` selects subtract
S	`0`	Preserve NZCV flags; `1` updates them
Fixed bits	`01011` and `0`	Assigned by Arm for this encoding class
`shift`	`00`	LSL; the absent shift defaults to LSL `#0`

Concatenate: `1 | 0 | 0 | 01011 | 00 | 0 = 10001011000`. The separate `imm6` field is `000000` for shift amount zero.

Common Data-Processing Register Patterns

All rows: **64-bit shifted-register forms**, `sf=1`, `shift=00` (LSL). Shift amount: `imm6`.

Arithmetic: fixed bits 28:24 = `01011`

Instruction	op / S	Prefix 31:21
add	0 / 0	10001011000
adds	0 / 1	10101011000
sub	1 / 0	11001011000
subs	1 / 1	11101011000

Logical: fixed bits 28:24 = `01010`

Instruction	opc	Prefix 31:21
and	00	10001010000
orr	01	10101010000
eor	10	11001010000
ands	11	11101010000

Arithmetic prefix: `sf | op | S | 01011 | shift | 0`.

Logical prefix: `sf | opc | 01010 | shift | N`, with `N=0` here.

For logical forms, opc is bits 30:29; `N` is bit 21. Setting `N=1` inverts the shifted second operand. For 32-bit versions, set `sf=0`; shifts must be 0–31.

Encoding Example: `add x5, x6, x7`

Register Numbers

$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$Rm = x7 = 00111$$

Operation Fields

operation = add

operand size = 64-bit

shift amount = 0



The prefix comes from `sf/op/S/fixed/shift`; now insert the operands:

`10001011000 00111 000000 00110 00101` = `0x8B0700C5`.

ADD/SUB: Immediate Form

Example Instruction

```
add x5, x6, #24
```



- The immediate field is 12 bits wide.
- The one-bit sh field (bit 22) selects a shift of 0 or 12 bits; bit 23 is fixed at 0.
- This format is common for address arithmetic and small integer constants.

Common Arithmetic Immediate Patterns

ADD/SUB (immediate) uses `sf | op | S | 100010 | sh | imm12 | Rn | Rd`. The six fixed bits `100010` occupy bits 28:23; `sh` is bit 22.

64-bit form	op	S	Prefix 31:23
`add Xd, Xn, #imm`	0	0	100100010
`adds Xd, Xn, #imm`	0	1	101100010
`sub Xd, Xn, #imm`	1	0	110100010
`subs Xd, Xn, #imm`	1	1	111100010

Derive the ADD Immediate Prefix

`sf=1, op=0, S=0: 1 | 0 | 0 | 100010 = 100100010`.
For `#24`, choose `sh=0` and encode 24 directly in `imm12`. With `sh=1`, the immediate is shifted left by 12 bits.

Encoding Example: `add x5, x6, #24`

Operands

$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\text{imm12} = 24 = 000000011000$$

Instruction Meaning

```
x5 = x6 + 24
```

The immediate value is part of the instruction word.



`add x5, x6, #24` is encoded as `0x910060C5`.

AArch64 Immediate Encodings Are Specialized

- Different instruction families have different immediate encodings.
- Some constants fit directly in an add or sub immediate.
- Other constants require move-wide instructions such as `movz`, `movn`, or `movk`.
- Logical instructions use their own bitmask-immediate encoding.

Immediate Fields

Do not ask whether a constant fits "in AArch64" in general. Ask whether it fits in the specific immediate field of the instruction being used.

Decoding Checklist

1. Convert the hexadecimal word to 32 binary bits.
2. Identify the broad encoding family from the opcode bits.
3. Split the word into the fields for that family.
4. Translate register fields into x or w register names.
5. Interpret immediates carefully: sign extension, scaling, and shifts matter.
6. Reconstruct the assembly instruction.

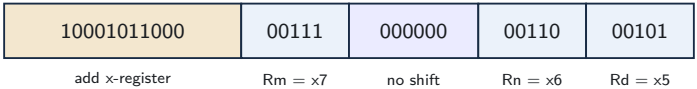
Important

The same bit positions can mean different things in different instruction families. Always identify the family before naming the fields.

Reverse Engineering Example

Given Machine Code

`0x8B0700C5`



Decoded Instruction

`0x8B0700C5` $\longrightarrow$ `add x5, x6, x7`

Section 4

Memory Instruction Encodings

Load/Store Instruction Encoding

Example Instruction

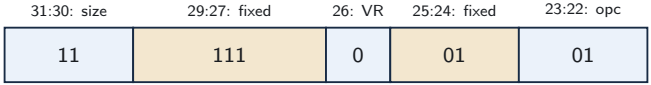
```
ldr x5, [x6, #16]
```



- `Rt` is the register loaded or stored.
- `Rn` is the base address register.
- `imm12` is a scaled unsigned offset for this common form.
- For a 64-bit `ldr`: address $=$ x6 $+ 8 \times$ imm12.

Deriving Load/Store Unsigned-Offset Prefixes

For integer **unsigned-offset** transfers, bits 31:22 are:



- `size`: `00`/`01`/`10`/`11` select 1/2/4/8-byte accesses for the transfers in the next table.
- `VR=0`: use general-purpose integer registers, rather than SIMD/FP registers.
- `opc=00`: store; `opc=01`: load without sign extension. Sign-extending loads use other opc values.
- `111` and `01` are ISA-assigned fixed bits for this form.

64-bit load: `11 | 111 | 0 | 01 | 01 = 1111100101`.
64-bit store: change opc to `00`, giving `1111100100`.

Common Memory Instruction Patterns

All rows use **integer unsigned-offset addressing**, `[Xn|SP, #offset]`, with `VR=0` and no base-register writeback.

Transfer	Bytes	size	opc	Prefix 31:22
`str Wt`	4	10	00	1011100100
`ldr Wt`	4	10	01	1011100101
`str Xt`	8	11	00	1111100100
`ldr Xt`	8	11	01	1111100101

Complete each word: `prefix | imm12 | Rn | Rt`. The encoded `imm12` equals byte offset divided by the number of bytes transferred.

Encoding Example: `ldr x5, [x6, #16]`

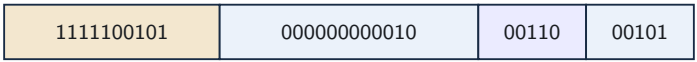
Operands

$$Rt = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\#16 = 2 \times 8 \text{ bytes}$$

Encoded Offset

$$\text{imm12} = 2 = 000000000010$$

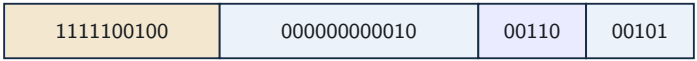
The instruction stores the scaled value, not the byte offset directly.



`ldr x5, [x6, #16]` is encoded as `0xF94008C5`.

Store Uses the Same Address Fields

`str x5, [x6, #16]` is encoded as `0xF90008C5`.



- The base register and scaled offset fields match the load example.
- opc (23:22) changes from `01` (load) to `00` (store); `size=11` remains unchanged.
- `Rt` is now the source register whose value is written to memory.

Addressing Mode Also Selects the Encoding

The mnemonic `ldr` alone does not determine the fixed bits. Compare these **64-bit integer load** forms:

Assembly form	Fixed leading bits	Offset / mode fields
`ldr Xt, [Xn, #off]`	`1111100101` (31:22)	`imm12`, scaled by 8
`ldr Xt, [Xn, #off]!`	`11111000010` (31:21)	signed `imm9`; 11:10 = `11`
`ldr Xt, [Xn], #off`	`11111000010` (31:21)	signed `imm9`; 11:10 = `01`

- Unsigned offset: no writeback. Pre-index `!`: update the base before access. Post-index: update it after access.
- In the two writeback forms, `imm9` (20:12) is a signed **byte** offset, -256 to 255. `Rn`/`Rt` remain in bits 9:5 / 4:0.
- For unsigned-offset `ldr`, bits 11:10 belong to `imm12`; they are not separate mode selectors.

Scaled Offsets Depend on Access Size

Unsigned Offset Form

$$\text{address} = \text{base} + \text{imm12} \times \text{access size}$$

Instruction kind	Access size	Encoded immediate meaning
`ldrb/strb`	1 byte	byte offset
`ldrh/strh`	2 bytes	byte offset divided by 2
`ldr w/str w`	4 bytes	byte offset divided by 4
`ldr x/str x`	8 bytes	byte offset divided by 8

Scaled Offset Range

Scaling lets a 12-bit field cover a larger byte range for naturally aligned accesses.

PC-Relative Addresses for Memory Access

- PC-relative instructions use the current instruction address as their reference; `adrp` uses its 4 KiB page base.
- This supports position-independent code and nearby constants or labels.
- For `adr` and literal loads, PC is the address of the current instruction.

Common PC-Relative Forms

- `adr`: forms an address near the current PC.
- `adrp`: forms a page address relative to the current PC page.
- `ldr literal`: loads data from a nearby PC-relative literal location.

Literal Loads in AArch64

Concept

A literal load reads a constant stored near the instruction stream using a PC-relative offset.



- The instruction encodes where to find the constant relative to the PC.
- The loaded value is data, even though it may be placed near the code.
- The pseudo-instruction `ldr x0, =constant` may use a literal pool; `ldr x0, literal` explicitly loads from a label.

Section 5

Branch Instruction Encodings

Common Immediate Branch Patterns

B/BL use a 6-bit prefix; conditional branches use 8 fixed high bits and a condition field.

Instruction	Fixed leading bits	Displacement field	cond (3:0)
`b label`	`000101` (31:26)	`imm26` (25:0)	—
`bl function`	`100101` (31:26)	`imm26` (25:0)	—
`b.eq label`	`01010100` (31:24)	`imm19` (23:5)	`0000`
`b.ne label`	`01010100` (31:24)	`imm19` (23:5)	`0001`
`b.lt label`	`01010100` (31:24)	`imm19` (23:5)	`1011`

- **B versus BL:** bit 31 selects branch (`0`) or branch-with-link (`1`); bits 30:26 stay `00101`.
- **Conditional branches:** bit 4 is fixed at `0`; cond selects EQ (Z=1), NE (Z=0), or signed LT (N differs from V).
- **All five rows:** target = current instruction address + sign-extended displacement $\times 4$. `bl` also writes PC + 4 to `x30`.

Unconditional Branch Encoding

Instruction Forms

```
b label        bl function
```



- `b` and `bl` use a signed 26-bit immediate field.
- The offset is measured in 4-byte instructions, not raw bytes.
- `bl` also writes the return address (PC + 4) into `x30`.

Target Calculation

$$\text{target address} = \text{PC} + \text{signextend}(\text{imm26}) \times 4$$

Branch Example: Forward Target

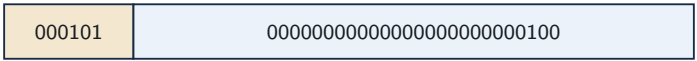
Reference PC = address of the branch itself. Encoded immediate = (target − PC) / 4.

Code Shape

```
0x100:  b target
0x104:  add x0, x0, #1
0x108:  sub x1, x1, #1
0x10c:  nop
0x110:  target:  nop
```

Offset

$$\Delta = \texttt{0x110} - \texttt{0x100}$$
$$= 16 \text{ bytes}$$
$$\text{imm26} = 16/4 = 4$$

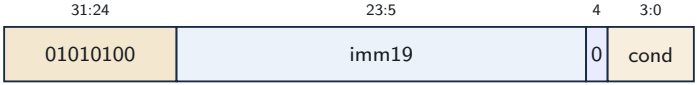


`b target`: positive displacement, `imm26=4`; word `0x14000004`.

Conditional Branch Encoding

Example Forms

```
b.eq label    b.lt label    b.ne label
```



- The condition code is encoded in the low 4 bits.
- The signed immediate is 19 bits, scaled by 4 bytes.
- Conditional branches test the flags set by earlier compare or arithmetic instructions.

Conditional Branch Example: `b.eq`

Target Offset

current PC $=$ `0x100`

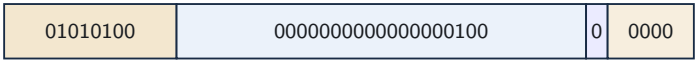
target $=$ `0x110`

imm19 $= 4$

Condition Field

eq $= 0000$

The branch is taken when the Z flag is set.



`b.eq target` is encoded as `0x54000080`.

Branch Example: Backward Loop

Assume `x0=0` and `x1` is a positive iteration count. The branch returns to an earlier instruction while `x1` is nonzero.

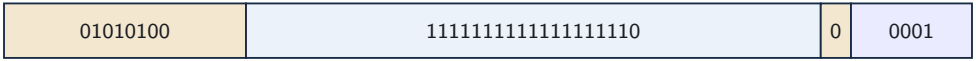
Loop Code (Instruction Addresses Shown)

```
0x200:  loop:  add x0, x0, #1
0x204:         subs x1, x1, #1
0x208:         b.ne loop
```

Signed Displacement

$$\text{PC} = \texttt{0x208}$$
$$\text{target} = \texttt{0x200}$$
$$\Delta = \texttt{0x200} - \texttt{0x208} = -8$$
$$\text{imm19} = -8/4 = -2$$

The 19-bit two's-complement representation of -2 is `1111111111111111110` (`0x7FFFE`). With `cond=0001` (NE):



Check: `0x208 + (-2 * 4) = 0x200`. Instruction word: `0x54FFFFC1`.

Common Encoding Mistakes

- Treating a register number as decimal text instead of a binary field.
- Forgetting that AArch64 register fields are usually 5 bits.
- Encoding a byte offset directly when the immediate field is scaled.
- Adding a pipeline offset to PC instead of using the branch instruction address.
- Reading little-endian memory bytes as if they were the printed instruction word.

Practical Habit

Write the bit ranges first, then fill in the fields. Do not try to guess the final hexadecimal word too early.

Topic 7: Summary

- AArch64 instructions are fixed-width 32-bit words.
- The meaning of each bit field depends on the instruction family.
- Fixed family patterns are looked up in the ISA; selector fields explain the related instruction variants.
- Data-processing immediate instructions use a different field layout.
- Load/store instructions commonly encode a base register, target/source register, and scaled offset.
- Direct immediate branches encode PC-relative offsets scaled by 4 bytes; register-indirect branches use a register target.
- In `b.cond`, bits 3:0 encode the condition and `imm19` encodes the signed instruction displacement.

Self-Test Questions: Instruction Words

1. How many bytes does an A64 instruction occupy? How far apart are consecutive instruction addresses?
2. If the instruction word is `0x8B020020`, what are its four bytes in increasing memory-address order?
3. Why must the instruction family be identified before interpreting operand fields?
4. What do `Rd`, `Rn`, and `Rm` represent in `add x5, x6, x7`? Give their 5-bit register numbers.
5. How do the fields change for `add x5, x6, #24`? What does its `sh` bit control?
6. Split `10001011000` into its selector and fixed fields. How does the prefix change for `subs x5, x6, x7`?

Self-Test Questions: Memory and Branches

1. In `ldr x5, [x6, #16]`, why is `imm12` equal to 2? Which field identifies the base register?
2. Derive `1111100101` from `size`, fixed bits, `VR`, and `opc`. What changes for `str x5, [x6, #16]`?
3. For a 32-bit unsigned-offset load with `imm12 = 3`, what is the byte offset?
4. What address is the reference for a literal load, and what page is the reference for `adrp`?
5. Compute the signed immediates for a branch at `0x100` to `0x110`, and a branch at `0x208` to `0x200`. Which PC is the reference?
6. Compare `b` and `b.eq`: give the immediate-field width, offset scaling, and location of the condition field where applicable.

Looking Ahead

Next Topic

Building a simple single-cycle RISC datapath.

- Instruction fields become inputs to the datapath and control logic.
- Register fields select operands in the register file.
- Immediate fields must be extracted, extended, and routed.
- Branch offsets feed the next-PC calculation.